

Password Storage Analysis Project Report

Comparative Analysis of Plain-Text, Hashed, and Compressed Password
Storage

Table of Contents

1. Introduction
2. Tools and Technologies Used
3. Project Design
4. Data Preprocessing
5. Data Storage Methods
6. Search and Performance Measurements
7. Results and Evaluation
8. References

1. Introduction

1.1 Purpose and Scope of the Project

This project investigates how different methods of storing the same dataset affect storage efficiency and search performance. A large-scale password dataset, obtained from the publicly available SecLists repository, is cleaned and standardized into a single reference dataset. This reference dataset is then stored using three distinct representations: plain text (Raw), one-way hash values (Hashed, using MD5 and SHA-256), and compressed binary data (Compressed, using the zlib library). For each representation, a custom C program performs random password lookups and measures disk usage, search latency, CPU usage, RAM usage, compression ratio, and decompression time.

The scope of the project covers the full pipeline: downloading and cleaning the raw dataset, organizing the project into a modular folder and code structure, implementing all three storage backends and their corresponding search routines, and finally measuring and interpreting their performance characteristics.

1.2 Motivation

Password storage is a practical example of a broader trade-off that appears throughout computer science: the trade-off between storage space, processing time, and security. Storing data as plain text is fast to search but consumes the most space and offers no protection if the underlying file is exposed. Hashing protects the original value but increases storage size (especially when multiple hash algorithms are stored per entry) and requires recomputing a hash for every search. Compression reduces disk usage significantly but introduces a decompression overhead before any data can be searched. Understanding these trade-offs quantitatively, rather than only in theory, is the central motivation of this project.

2. Tools and Technologies Used

2.1 Programming Language

The entire project is implemented in the C programming language, as required by the project specification. No C++ features were used.

2.2 Development Environment

The project was developed and compiled on Windows using the MSYS2 environment with the MinGW-w64 GCC toolchain. MSYS2 was chosen because it provides a Unix-like shell together with a package manager (pacman), which simplifies the installation of native C libraries such as zlib that are not available by default on Windows.

- Operating System: Windows 10/11
- Compiler: GCC 16.1.0 (MinGW-w64, via MSYS2)

- Shell: MSYS2 MINGW64 terminal
-
- Build command: a single gcc invocation compiling all modules together (no separate build system was required given the project size)

2.3 Libraries Used

- zlib — used for the Compressed storage backend (gzopen/gzwrite/gzread family of functions) and for compression ratio / decompression time measurement.
- Windows API (windows.h, psapi.h) — used to measure CPU usage (GetSystemTimes) and RAM usage (GetProcessMemoryInfo) during the search benchmarks.
- Public-domain MD5 and SHA-256 implementations (Brad Conte, crypto-algorithms) — used for the Hashed storage backend.
- Standard C library (stdio.h, stdlib.h, string.h, time.h, math.h) — used for file I/O, string handling, high-resolution timing, and statistical calculations.

•

3. Project Design

3.1 Folder Structure

The project follows a modular folder structure that separates raw input data, processed data, the three storage representations, source code, results, and the report itself. This separation ensures that each stage of the pipeline has a single, unambiguous responsibility.

Folder	Purpose
RawData/	Stores the original, unmodified password files downloaded from the SecLists repository.
Processed/	Stores the cleaned, deduplicated, and standardized dataset (passwords_clean.txt) used as the single reference dataset for all storage methods.
Storage/Raw/	Plain-text representation of the processed dataset, used as the performance baseline.
Storage/Hashed/	Stores MD5 and SHA-256 hash values of each password instead of the plain text.
Storage/Compressed/	Stores the processed dataset compressed with the zlib library (.gz format).
Code/	Contains all C source files, organized into modules by responsibility.

Results/	Stores the automatically generated performance results (results.txt).
Report/	Contains this report document.

3.2 Data Flow

The data flows through the system in a strictly one-directional pipeline, as required by the project specification:

1. Raw password files are downloaded into RawData/ and are never modified directly.
2. preprocess.c reads the files in RawData/, removes empty lines and duplicate entries, and writes a single standardized file to Processed/.
3. storage.c reads exclusively from Processed/ — never from RawData/ — and produces three independent representations of the same dataset in Storage/Raw/, Storage/Hashed/, and Storage/Compressed/.
4. search.c performs lookups directly against each of the three storage representations.
5. main.c orchestrates steps 1–4, selects a random sample of passwords from the processed dataset, runs the benchmark for each storage method, and writes the results to Results/results.txt.

This design guarantees that all three storage methods are built from exactly the same underlying data, which is essential for a fair performance comparison.

3.3 Code Modules

Rather than implementing the entire project in a single file, the source code is split into modules according to responsibility, as recommended in the project description. Each .c file corresponds to one stage of the pipeline or one supporting utility.

File	Responsibility
main.c	Entry point of the program. Coordinates preprocessing, storage, random sample selection, benchmarking, and printing of results.
preprocess.c	Reads the raw input files, removes empty lines and duplicate passwords using a hash-table-based duplicate check, and writes the cleaned dataset to Processed/.
storage.c	Writes the processed dataset into the three storage representations: Raw (plain text), Hashed (MD5 + SHA-256), and Compressed (zlib).
search.c	Implements the search routines for each storage method: linear search for Raw, hash comparison for Hashed, and search-after-decompression for Compressed.
compression.c	Implements compression-ratio calculation and measures decompression time using zlib.
utils.c / utils.h	Provides shared helper functions: high-resolution timing (clock_gettime), file size retrieval, CPU usage calculation (GetSystemTimes), RAM usage measurement (GetProcessMemoryInfo), and formatted result logging.
md5.c / md5.h / md5_wrapper.c	Third-party MD5 implementation (Brad Conte, public domain) wrapped in a simple md5_string() function used by storage.c and search.c.
sha256.c / sha256.h / sha256_wrapper.c	Third-party SHA-256 implementation (Brad Conte, public domain) wrapped in a simple sha256_string() function, used as the optional additional hash required by the project description.

4. Data Preprocessing

3.4 Reading the Raw Data

Two password list files from the SecLists Common-Credentials collection were selected as the raw dataset for this project: Pwdb_top-10000000.txt and xato-net-10-million-

passwords1000000.txt. Both files are read line by line using `fgets()`, and trailing carriage-return / newline characters are stripped from each line before further processing.

Source File	Raw Lines	After Cleaning
Pwdb_top-10000000.txt	511,733	450,747
xato-net-10-million-passwords-1000000.txt	999,999	472,437

3.5 Removing Duplicate Records

Empty lines are discarded immediately. Duplicate detection is performed using a custom hash table: each password string is hashed with the djb2 string-hashing algorithm into a table of just over two million buckets, and a password is only written to the output file if its bucket has not already been marked as occupied. This approach was chosen because it processes each line in constant time on average, which is necessary given that the combined raw input exceeds 1.5 million lines.

No case normalization was applied in this project; passwords are treated as case-sensitive strings exactly as they appear in the source files. This decision is noted explicitly here in accordance with the project requirements.

3.6 Producing the Standardized Dataset

After both source files are processed (the second file is appended to the output of the first), the result is a single file, `Processed/passwords_clean.txt`, containing 923,184 unique, non-empty password entries in one consistent format: one password per line, with no leading or trailing whitespace. This file is the only data source used by every storage method in the next stage.

3.7 Hash Generation

Hash values are not generated during preprocessing; they are generated during the storage stage, directly from the cleaned dataset. For every password in `Processed/passwords_clean.txt`, two hash digests are computed: a 128-bit MD5 digest (32 hexadecimal characters) and a 256-bit SHA-256 digest (64 hexadecimal characters). Both algorithms were implemented using publicdomain reference implementations and wrapped in `simple_md5_string()` and `sha256_string()` helper functions that take a password and return its hexadecimal digest.

4. Data Storage Methods

4.1 Raw Storage

The Raw storage method copies the processed dataset into `Storage/Raw/passwords.txt` without any transformation. Each password is written exactly as it appears in the processed file, with no additional encoding or compression. This representation serves as the performance baseline against which the Hashed and Compressed methods are compared, since it reflects the cost of working with the data in its most direct form.

4.2 Hashed Storage

The Hashed storage method never writes the plain-text password to disk. Instead, for every entry it computes both an MD5 digest and a SHA-256 digest and writes them, space-separated, to `Storage/Hashed/passwords_md5.txt`. Storing both digests was an optional addition (the project specification requires at least MD5) included to allow a direct comparison between a faster, shorter, legacy hash function (MD5) and a slower, longer, modern hash function (SHA-256).

Searching against the Hashed storage works by first hashing the user-supplied target password and then performing a string comparison against the MD5 portion of each stored line, since MD5 is the faster of the two algorithms to compute and compare.

4.3 Compressed Storage

The Compressed storage method writes the entire processed dataset into `Storage/Compressed/passwords.gz` using the zlib library's gzip-compatible interface (gzopen with the "wb9" mode for maximum compression level, followed by gzwrite). Because gzip-compressed data cannot be searched directly, every lookup against this storage method first streams and decompresses the file using `gzgets()` and then performs a standard string comparison line by line.

5. Search and Performance Measurements

5.1 How Searches Are Performed

For each storage method, a dedicated search function is implemented in `search.c`:

- `search_raw()` performs a linear scan of the plain-text file, comparing each line directly against the target password.
- `search_hashed()` computes the MD5 digest of the target password once, then performs a linear scan of the hashed file, comparing the stored MD5 portion of each line against the computed digest.

- `search_compressed()` opens the compressed file with `gzopen()`, decompresses it on the fly line by line with `gzgets()`, and compares each decompressed line against the target password.
-

5.2 Sampling and Timing Methodology

A pool of up to 500,000 passwords is loaded from the processed dataset, and 1,000 passwords are then selected at random from this pool using a fixed random seed (42), which ensures that the same sample is used consistently across repeated runs of the program. This sample size exceeds the project's minimum requirement of 100 searches by a factor of ten, in order to obtain a more statistically stable average and standard deviation.

For every individual search, the elapsed time is measured using `clock_gettime(CLOCK_MONOTONIC, ...)`, which provides microsecond-level resolution and is not affected by changes to the system clock. The 1,000 individual timings are then used to compute the average search time, the sample standard deviation, and the maximum (worst-case) search time for each storage method.

5.3 CPU and Memory Measurement

CPU usage during each benchmark is measured using the Windows API function `GetSystemTimes()`, which reports cumulative kernel and idle time for the whole system immediately before and immediately after the 1,000 searches; the percentage of non-idle time during that interval is reported as the CPU usage for that storage method. RAM usage is measured using `GetProcessMemoryInfo()`, which reports the working-set size of the running process in megabytes at the end of each benchmark.

5.4 Disk Size Measurement

Disk usage for each storage method is measured directly from the file system using the `stat()` system call on the corresponding output file (`Storage/Raw/passwords.txt`, `Storage/Hashed/passwords_md5.txt`, and `Storage/Compressed/passwords.gz`), and reported in megabytes.

6. Results and Evaluation

6.1 Summary Results Table

The table below summarizes the four core metrics requested by the project specification — disk usage, average search time, maximum search time, and the underlying method — for all three storage backends, based on 1,000 random searches against the processed dataset of 923,184 unique passwords.

Storage Method	Avg. Search Time	Max. Search Time	Disk Usage
Raw	824.62 μ s	1930.40 μ s	8.88 MB
Hashed (MD5+SHA256)	4654.19 μ s	16400.50 μ s	91.40 MB
Compressed (zlib)	1159.88 μ s	2459.50 μ s	4.10 MB

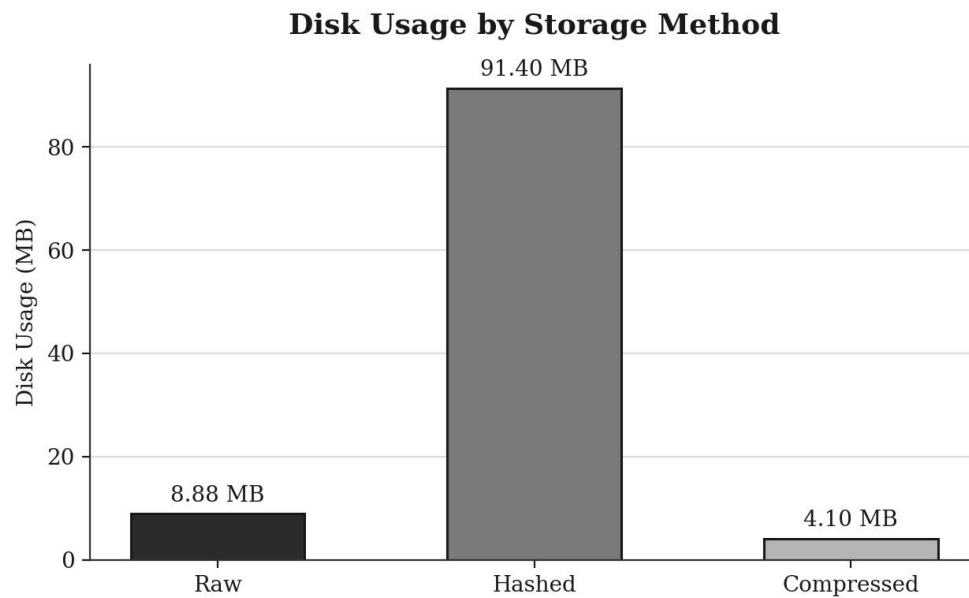


Figure 1. Disk usage (MB) for each storage method.

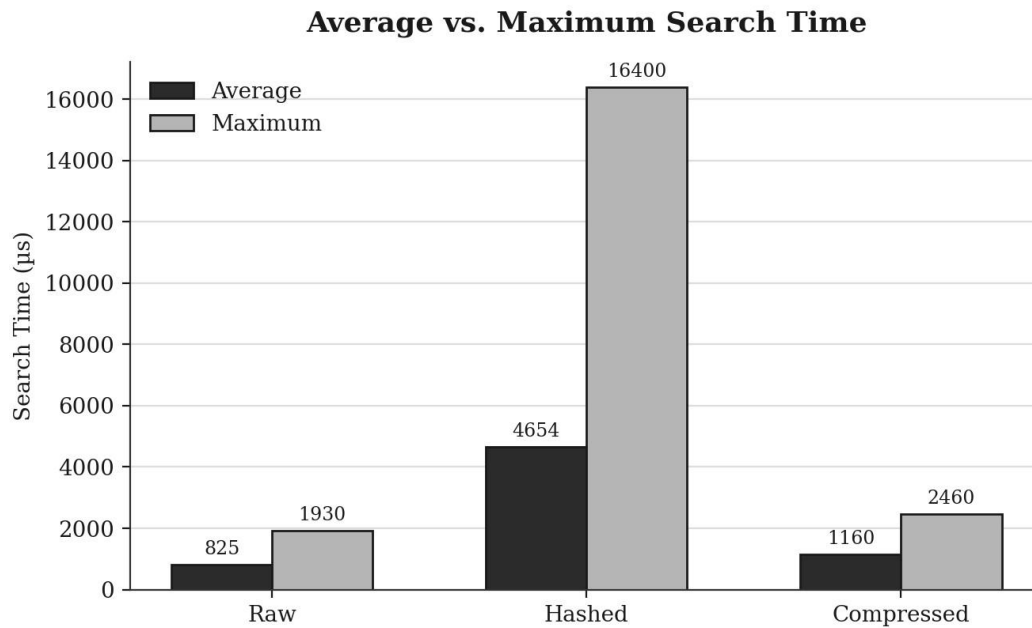


Figure 2. Average and maximum search time (μ s) for each storage method.

6.2 Full Performance Metrics

The following table presents the complete set of measured metrics, including standard deviation, CPU usage, and RAM usage, for each storage method.

Method	Avg. (μ s)	Std. Dev. (μ s)	Max. (μ s)	CPU (%)	RAM (MB)
Raw	824.62	483.98	1930.40	10.92	12
Hashed	4654.19	2846.34	16400.50	14.99	12
Compressed	1159.88	671.38	2459.50	10.89	12

6.3 Compression-Specific Metrics

- Compression Ratio: 0.4615 (original size 8.88 MB, compressed size 4.10 MB)
- Decompression Time: 39,590.70 μ s for the full dataset

**Compressed Size vs. Original Size
(Compression Ratio = 0.4615)**

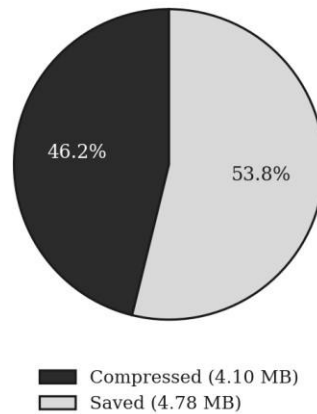


Figure 3. Proportion of the original dataset size retained after zlib compression.

6.4 Discussion

Disk Usage

Compressed storage uses the least disk space by a wide margin, reducing the dataset to under half its original size (a compression ratio of 0.46). Raw storage is the second most space-efficient, since it stores each password exactly once with no additional encoding. Hashed storage uses more than ten times the disk space of Raw storage, because every password is replaced by two fixed-length hash digests (32 and 64 hexadecimal characters respectively) that are, on average, longer than the original password strings — and because two separate algorithms (MD5 and SHA256) are stored per entry rather than one.

Search Speed

Raw storage provides the fastest average search time, since a lookup only requires reading and comparing plain-text lines with no additional computation. Compressed storage is roughly 1.4 times slower than Raw on average, reflecting the cost of decompressing the file before each comparison can take place. Hashed storage is by far the slowest, at more than five times the average search time of Raw storage, because every single lookup requires computing a fresh MD5 digest of the target password before any comparison can begin, and this hashing cost dominates the search time.

Worst-Case Behavior

The maximum search time follows the same ordering as the average: Hashed storage has by far the largest worst-case latency (16.4 ms), more than eight times that of Raw storage. This indicates that the performance gap between Hashed and the other two methods is consistent across both typical and worst-case scenarios, rather than being driven by a few outliers.

CPU and RAM Usage

CPU usage is broadly comparable across the three methods, in the range of roughly 11% to 15% of the system during the benchmark window, with Hashed showing a slightly higher figure consistent with the additional computation required for hashing. RAM usage is essentially constant at 12 MB across all three methods, indicating that the working-set footprint of the search routines themselves is small and is not a distinguishing factor between the storage strategies for this dataset size.

Compression and Decompression Overhead

The 0.46 compression ratio confirms that zlib is effective at reducing the footprint of this dataset, despite passwords being relatively short, high-entropy strings that do not compress as well as natural-language text. The roughly 40 ms decompression time, when amortized across many searches (as would happen in a real application that decompresses once and then performs many lookups against the in-memory result, rather than decompressing per search as this benchmark does), would represent a small one-time cost rather than a per-search penalty.

6.5 Advantages and Disadvantages

Raw Storage

- Advantages: fastest search time; simplest implementation; no computational overhead.
- Disadvantages: no confidentiality — anyone with file access can read every password directly; moderate disk usage compared to Compressed storage.

Hashed Storage

- Advantages: the original password is never stored, which is the standard practice for realworld credential storage; supports verification ("does this password match?") without ever exposing the plain text.
- Disadvantages: substantially larger disk footprint, especially when multiple hash algorithms are stored per entry; significantly slower search, since a hash must be computed for every single lookup; does not support partial or fuzzy matching, since any change to the input completely changes the hash.

Compressed Storage

- Advantages: lowest disk usage by far among the three methods; still allows the original data to be fully recovered (unlike hashing).
- Disadvantages: requires a decompression step before any search can be performed, adding latency; offers no confidentiality on its own, since compressed data can be decompressed by anyone who obtains the file.

6.6 Overall Comparison

No single storage method is superior on every metric: Raw storage is best for search speed, Compressed storage is best for disk efficiency, and Hashed storage is the only method that protects the confidentiality of the original passwords, at the cost of both space and speed. In a real system, the appropriate choice depends on the requirement that matters most — for example, a production authentication system would use a hashed (and ideally salted) representation despite its overhead, because protecting user passwords outweighs the performance cost, whereas a system primarily concerned with minimizing storage cost for non-sensitive data would prefer compression.

7. References

- Miessler, D. SecLists — Common-Credentials password lists. GitHub repository: <https://github.com/danielmiessler/SecLists/tree/master/Passwords/Common-Credentials>
- zlib — A Massively Spiffy Yet Delicately Unobtrusive Compression Library. Official documentation: <https://zlib.net/manual.html>
- Conte, B. crypto-algorithms — public-domain C implementations of MD5 and SHA-256. GitHub repository: <https://github.com/B-Con/crypto-algorithms>
- Microsoft Learn. GetSystemTimes function (processthreadsapi.h). Windows API documentation.
- Microsoft Learn. GetProcessMemoryInfo function (psapi.h). Windows API documentation.
- Microsoft Learn. clock_gettime and high-resolution timing in C / POSIX threads documentation (time.h).